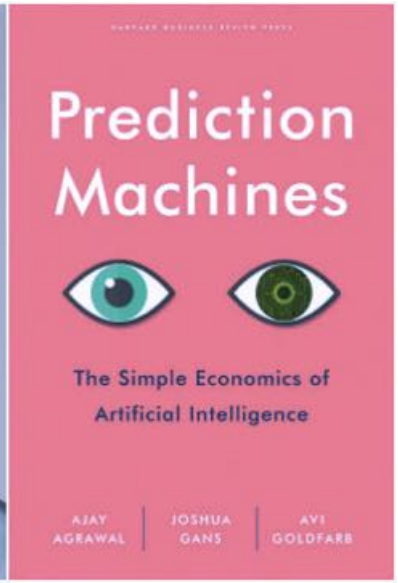
Regularization
in Machine Learning

Machine Learning Systems = Prediction Machines



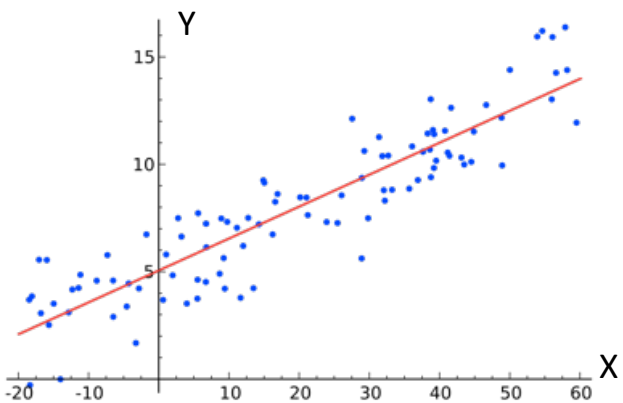
Ajay Agrawal
University of Toronto

ML = "Cheap Predictions"

AI = "Cheap Predictions"

OK, so what is Machine Learning Model?

.. A computer program, a function
.. used to describe data in a compressed way
.. to make predictions and take actions
.. most common ML tasks - regression or classification



Linear Regression y = f(x) = ax + b
Model uses only two numbers:
  a = slope, b = intercept
Model compresses hundreds of numbers into just two numbers to make "cheap predictions".

Regress vs Progress.

Regress = simplify to the core, to the basics.

Regression - develop a simple model (based on training data) so that this simple model can do predictions.

Model - a function that returns prediction based on input arguments.

==========

Example 1 - Linear Regression
 (draw straight line through points).
 Model function uses two parameters [a,b] under the hood:

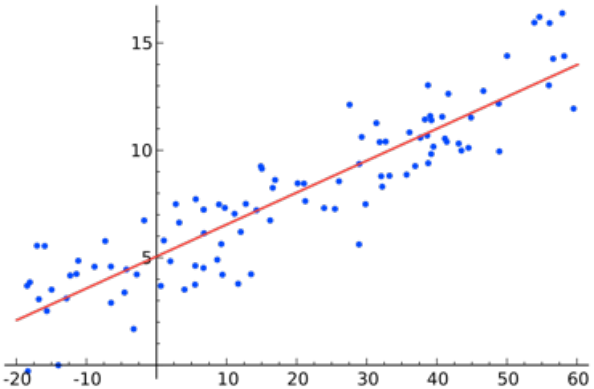
```
def linear_regression(x):
  return (a*x + b)
```

Training data - array of data points (x,y)

Training of the Linear Regression model means simply finding two numbers (a=slope, b=intercept) which do the best fit between the model and training data (minimize the error between model and data).

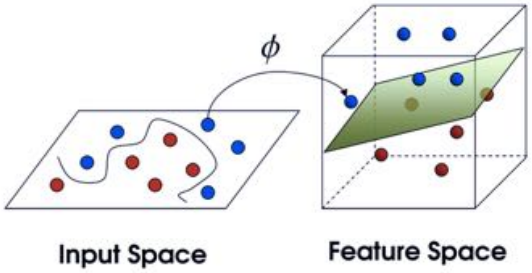
Different Types of Machine Learning Models

Simple Analytics



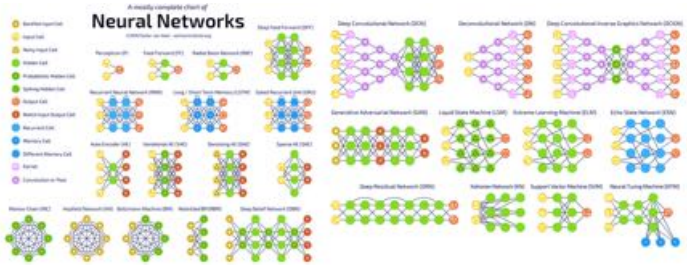
Linear Regression y = f(x) = ax + b
Model uses only two numbers:
a = slope, b = intercept

Standard ML



Classification - learning to label examples based on their features. Model may have ~1-2 thousand of numbers

Logistic Regression
Linear Discriminant Analysis
Support Vector Machines
Decision trees (Random Forest, XGBoost)
etc.



Neural Networks (Deep Learning)

Example: classification – dog or cat.
Model is usually implemented as an Artificial Neural Network (Deep Learning).
Large ANNs can have up to 170 Bln parameters.
Examples: recognizing images, music, speech, NLP (Natural language Processing), Machine Translation, supervised, unsupervised, reinforcement learning, Generative Models (GANs)

How to find "REGULARITY"

Overfitting and Regularization

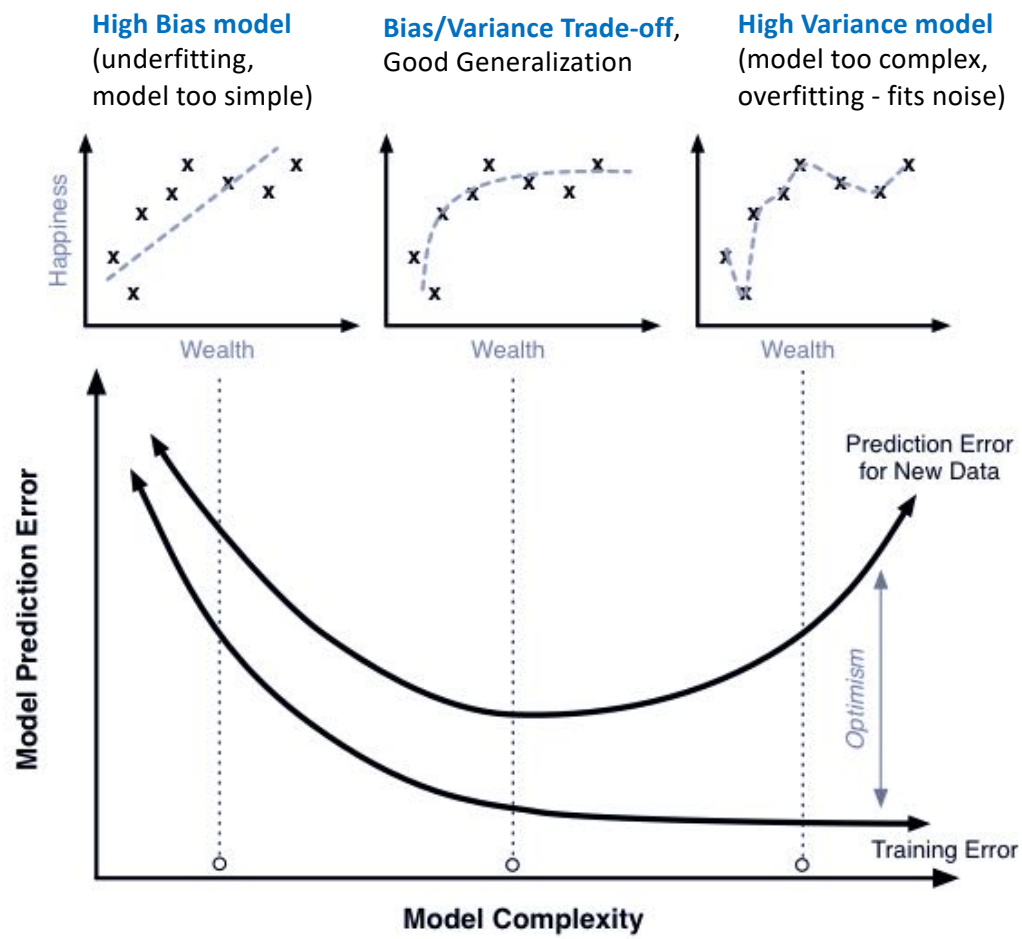
Goal of training the model is to fit/learn a "**regularity**" in the data, and to avoid fitting noise (avoid overfitting).

How do we know that we are overfitting?

Let's split data into two sets:
- training
- testing

If we **overfit** on the training data (learn regularity and some noise specific to the training data), then accuracy measured on the test data may be bad (model doesn't generalize well).

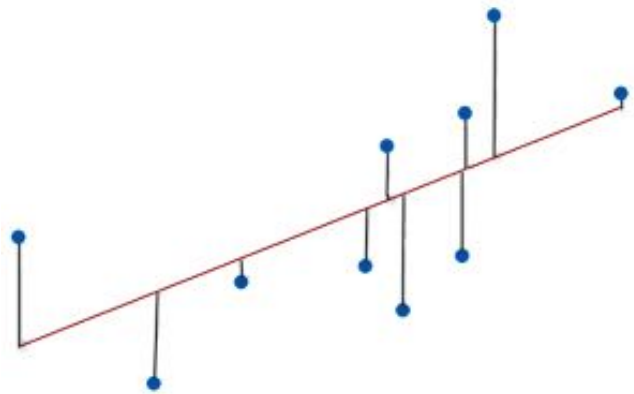
Solution – try to find a more "**regular**" solution (less noise, less overfitting).



We fit best model (best line) to minimize the error, the distance between the model and the data

$$\mathrm{SSE} = \underbrace{\sum_{i=1}^{n}}_{\text{test set}} (\underbrace{y_i}_{\text{predicted vaue}} - \underbrace{\hat{y}_i}_{\text{actual value}})^2$$

SSE = Sum of Squares Error



Regularization: Ridge Regression

Multivariate (multiple) Linear Regression Regularization.

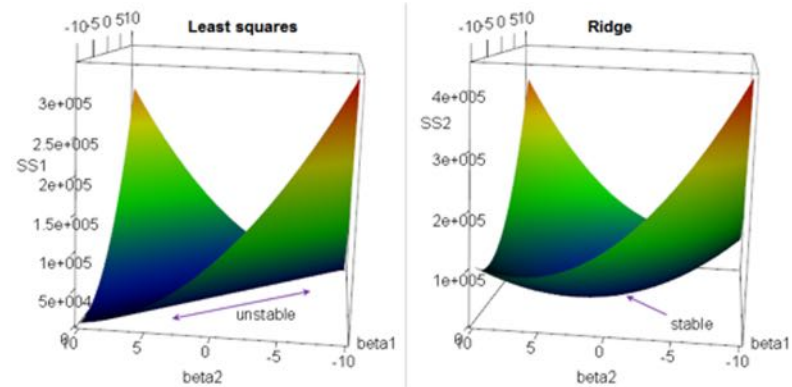
Suppose we have a OLS (Ordinary Linear Squared) regression where two parameters x1 & x2 are not orthogonal, but are equal, proportional to each other, or highly correlated. Then there may be an infinite number of coefficients which will fit the model.

For example, (a*x1 - x2), (100*a*x1 - 100*x2), etc.

If we are trying to minimize the cost function in the multidimensional space of coefficients, we will get an infinite diagonal **groove (ridge)** for a*x1 = x2. Any place at the bottom of this ridge is equally good for us. So we will get a big variability in possible values of coefficients for x1 & x2. In other words, the solution will be unstable.

We can reduce this variability by adding additional regularization factor to the loss function as a sum of squares of coeffitients. When we minimizing this expression, bigger "lambda" value will cause smaller "beta" values, thus keeping the coefficients "beta" from becoming too large.
This is called "**Ridge Regression**".
It changes the infinitely long ridge into a local minimum.

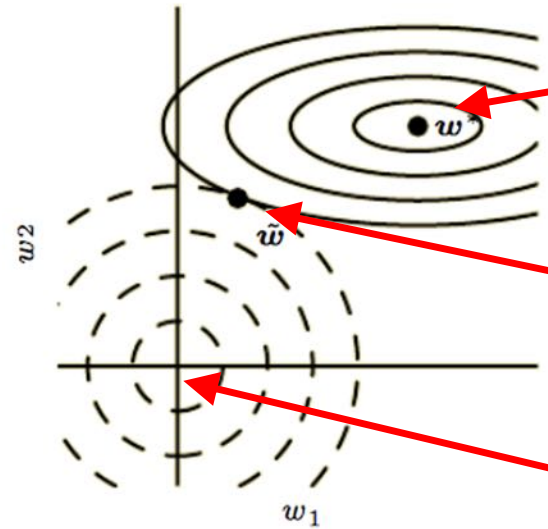
Why Ridge Regression is called Ridge ...



$$\sum_{i=1}^{n}\left(y_i-\beta_0-\sum_{j=1}^{p}\beta_j x_{ij}\right)^2+\lambda\sum_{j=1}^{p}\beta_j^2=\mathrm{RSS}+\lambda\sum_{j=1}^{p}\beta_j^2$$

One way to do avoid overfitting can be to avoid big "weights" in the model.

$$Cost\,function = Loss + \frac{\lambda}{2m} * \sum \|w\|^2$$



Minimum "cost" without regularization

Compromize
min(cost + penalty)
Cons: bias (shift),
Pros: low variance

Minimum of the "penalty" regularization term

Figure 7.1: An illustration of the effect of L^2 (or weight decay) regularization on the value of the optimal $\boldsymbol{w}$. The solid ellipses represent contours of equal value of the unregularized objective. The dotted circles represent contours of equal value of the L^2 regularizer. At the point $\tilde{\boldsymbol{w}}$, these competing objectives reach an equilibrium. In the first dimension, the eigenvalue of the Hessian of J is small. The objective function does not increase much when moving horizontally away from $\boldsymbol{w}^*$. Because the objective function does not express a strong preference along this direction, the regularizer has a strong effect on this axis. The regularizer pulls w_1 close to zero. In the second dimension, the objective function is very sensitive to movements away from $\boldsymbol{w}^*$. The corresponding eigenvalue is large, indicating high curvature. As a result, weight decay affects the position of w_2 relatively little.

Regularization - LASSO, Elastic Net

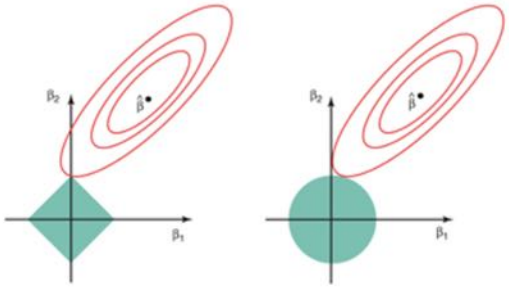
In **Ridge Regression** we add a "**regularization**" term as **sum of squares** (**L2 norm**).

In **LASSO Regression** we add a "**regularization**" term as **sum of abs values** (**L1 norm**).

LASSO = Least Absolute Shrinkage and Selection Operator.

LASSO favors solutions on the tips of the bounding surface for coefficients, where some of coefficients are zero. Thus it favors "sparse" solutions.

Elastic Net - A combination of Ridge Regression and LASSO.



Why LASSO favors sparse solutions

7.1. Parameter Norm Penalties

Regularization has been used for decades prior to the advent of deep learning. Linear models such as linear regression and logistic regression allow simple, straightforward, and effective regularization strategies.

Many regularization approaches are based on limiting the capacity of models, such as neural networks, linear regression, or logistic regression, by adding a parameter norm penalty $\Omega(\boldsymbol{\theta})$ to the objective function J. We denote the regularized objective function by $\tilde{J}$:

Parameter norm penalty

$$\tilde{J}(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y}) = J(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y}) + \alpha\Omega(\boldsymbol{\theta}) \tag{7.1}$$

where $\alpha \in [0, \infty)$ is a hyperparameter that weights the relative contribution of the norm penalty term, Ω, relative to the standard objective function J. Setting α to 0 results in no regularization. Larger values of α correspond to more regularization.

Note: θ – some (sub)set of parameters.

Note: loss function is a part of a cost function which is a type of an objective function

Loss function – measures the penalty

Cost function – more general (may include regularization)

Objective function – any function you optimize during training

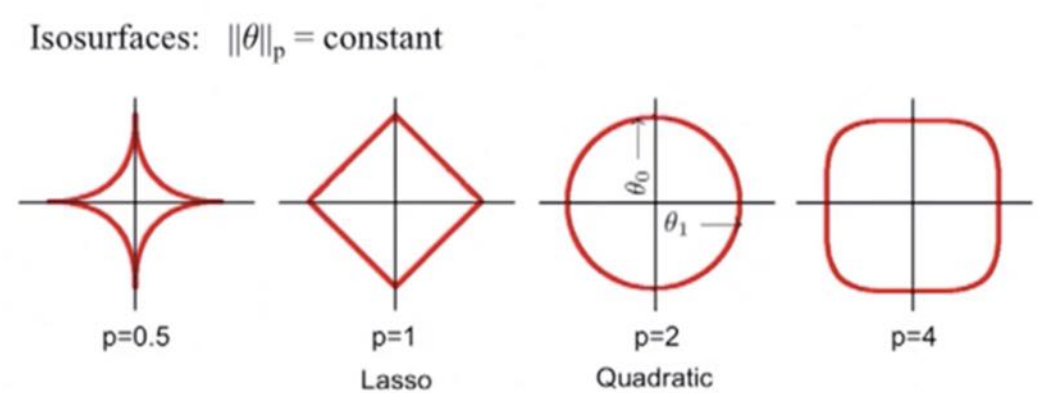
L1 & L2 Regularization

L1 norm - minimizing the sum of the absolute differences

L2 norm - minimizing the sum of the square of the differences

$$\text{L1:} \quad R(\theta) = \|\theta\|_1 = \sum_{i=1}^{n} |\theta_i|$$

$$\text{L2:} \quad R(\theta) = \|\theta\|_2^2 = \sum_{i=1}^{n} \theta_i^2$$



L^2 Parameter Regularization

L^2 parameter norm penalty commonly known as:
"**weight decay regularization**"
or "**ridge regression**"
or "**Tikhonov regularization**".

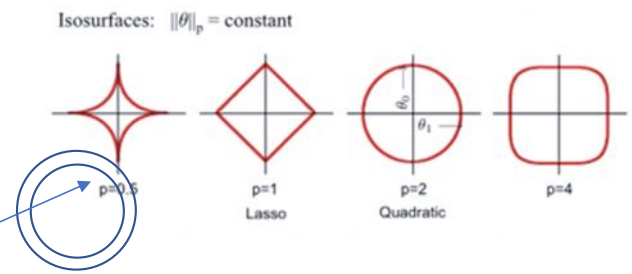
This regularization strategy drives the weights closer to the origin
by adding a quadratic regularization term to the objective function.
… the addition of this weight decay term has modified the learning rule
to multiplicatively shrink the weight vector by a constant factor on each step, just
before performing the usual gradient update.

L^1 Regularization

$$\Omega(\boldsymbol{\theta}) = ||\boldsymbol{w}||_1 = \sum_i |w_i|$$

Regularized objective function:

$$\tilde{J}(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y}) = \alpha ||\boldsymbol{w}||_1 + J(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y})$$



In comparison to L^2 regularization,
L^1 regularization results in a solution that is more **sparse**
(**sparse** = some parameters have an optimal value of zero).

L^2 regularization does not cause the parameters to become sparse,
while L^1regularization may do so for large enough **α**.

The sparsity property induced by L^1 regularization has been used extensively
as a **feature selection mechanism**.
Feature selection simplifies a machine learning problem
by choosing which subset of the available features should be used.
In particular, the well known **LASSO** (Tibshirani, 1995) (Least Absolute Shrinkage and Selection Operator) model
integrates an L^1 penalty with a linear model and a least squares cost function.
The L^1 penalty causes a subset of the weights to become zero,
suggesting that the corresponding features may safely be discarded.

Regularization:

"any modification we make to a learning algorithm
that is intended to reduce its **generalization error**
but not its training error."

Motivation:
Imagine that we train network on different subsets of inputs.
For each subset we will get different model parameters.
We want to reduce the variance of these parameters between subsets.

generalization vs overfitting

"… most regularization strategies are based on regularizing estimators ...
by trading increased bias for reduced variance.
An effective regularizer is one that makes a profitable trade,
reducing variance significantly while not overly increasing the bias."

Regularization - Bagging and Other Ensemble Methods

- Bagging (short for **bootstrap aggregating**) is a technique for reducing generalization error.
- This is done by separately training several different models and then combining them. Each model then votes on what it thinks the best output is.
- A general term for this is **model averaging**.
- Bagging is a type of **ensemble method**.
- The reason we use bagging is that different models often have varying degrees of generalization and test error.
- Machine Learning contests are often won using methods of **model averaging** over dozens of models.

Regularization – Generating Data (Dataset Augmentation)

Idea – generate fake data and add it to the training data set.

Example for images:
.. shift images several pixels in all directions,
.. slight rotation,
.. mirror reflection,
.. slight scaling.

be careful – certain transformation can change the meaning:
‘**b**’ : ‘**d**’ ,
‘**6**’ : ‘**9**’

Example - training AlphaGo Zero - playing Chess against itself.

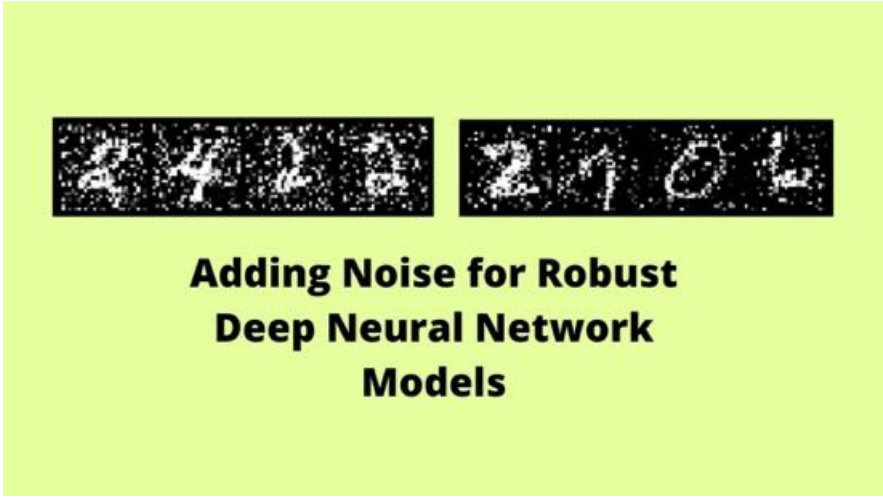
Example - training self-driving car algorithm using simulator (driving inside randomly generated virtual world).

Regularization by **Injecting noise**

This is very common technique.

It is another form of data augmentation.

Can be applied to:
.. the inputs,
.. the hidden units
.. the weights (dropout),
.. output targets.



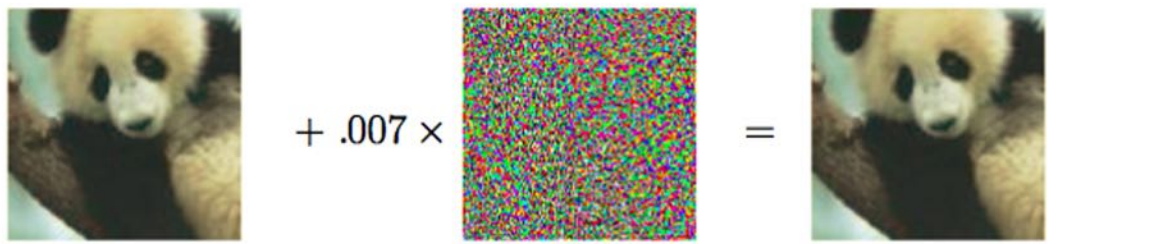
Regularization – Random Dropout

(adding noise into training process)

- Dropout - a regularization technique for neural network models proposed by Srivastava, et al, 2014. Dropout is a technique where randomly selected neurons are ignored during training. They are "dropped-out" randomly (by specifying a random mask vector).
- Dropout provides an inexpensive approximation to training and evaluating a bagged ensemble of exponentially many neural networks. Typically, Dropout can have good performance with only 10-20 of its exponentially many models that are averaged together using the geometric mean of the output.
- Weight Scaling Inference Rule: Make sure the expected total input to a unit at test time is roughly the same as the expected total input to that unit at train time. One example is randomly removing any given weight from the model 50% of the time and then dividing the values of each weight by 2 at end of training or multiply the weights by 2 during training.
- Dropout is computationally cheap and usually more effective than other standard computationally inexpensive regularization methods such as weight decay, filter norm constraints and sparse activity regularization.
- Dropout may also be combined with other forms of regularization to yield a further improvement.
- Dropout works well with nearly any model that uses a distributed representation and can be trained with stochastic gradient descent.

Regularization - Adversarial Training

- Problem: if a network consists of primarily linear elements, small changes to the input data may cause large errors in the output.
- Solution: train your network using injected randomness.



$\boldsymbol{x}$	$\text{sign}(\nabla_{\boldsymbol{x}} J(\boldsymbol{\theta}, \boldsymbol{x}, y))$	$\boldsymbol{x} + \epsilon\,\text{sign}(\nabla_{\boldsymbol{x}} J(\boldsymbol{\theta}, \boldsymbol{x}, y))$
$y =$"panda" w/ 57.7% confidence	"nematode" w/ 8.2% confidence	"gibbon" w/ 99.3 % confidence

Figure 7.8: A demonstration of adversarial example generation applied to GoogLeNet (Szegedy *et al.*, 2014a) on ImageNet. By adding an imperceptibly small vector whose elements are equal to the sign of the elements of the gradient of the cost function with respect to the input, we can change GoogLeNet's classification of the image. Reproduced with permission from Goodfellow *et al.* (2014b).

Regularization - Early Stopping.

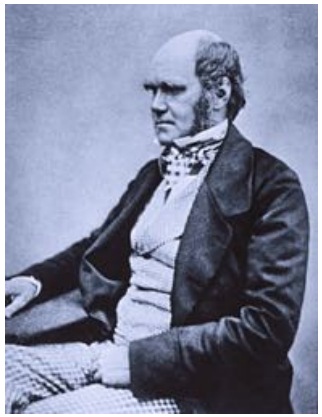
When training large models with sufficient representational capacity to overfit the task, we often observe that training error decreases steadily over time,
but validation set error begins to rise again.

Every time the error on the validation set improves, we store a copy of the model parameters, so that we can go back to them and stop training.
This is called "**Early stopping**".

Early stopping requires a validation set, which means some training data is not fed to the model. To best exploit this extra data, one can perform extra training after the initial training with early stopping has completed. In the second, extra training step, all of the training data is included.

Regularization – Evolutionary (Meta) Learning

Meta Learning = learn to learn



Charles Darwin

For example, automatically find the best model architecture and hyperparameters (Meta-parameters).
Search for optimum in a multi-dimensional space of architectures and hyperparameters (Meta-Space).
We can optimize for different things - accuracy, generalization, speed, power-effectiveness, etc.

Random search is slow.
Evolutionary approach is much faster.
It is similar to Gradient Descent in the space of hyper-parameters.
Idea – let models change and compete, select the best, repeat.
Similar to Darwinian evolution in biology.

Examples of random "mutations": remove a convolution, add skip connection, change learning rate, etc.
Combinations - analogous to genetic recombination - combining genes (hyperparameters) of two parents.

Many groups are working on the subject:
- OpenAI (Reptile, MAML)
- Uber Labs (differentiable plasticity)
- Sentient Labs (evolutionary optimization of gated networks (LSTMs) for language modeling)
- Google DeepMind
- Google Brain AutoML, Amoebanet, AdaNet
- H2O.ai - AutoML and Driverless AI